# Project Description Report

## Project Overview

**Project Name:** Harvard Processor Simulator
**Package Number:** Package 4
**Package Name:** Double McHarvard with cheese circular shifts

## Team Information

**Team Number:** 63
**Team Name:**

### Team Members

| Name | ID | Tutorial |
|---|---|---|
| Adham Jalaal Khalaaf | 61-11064 | T25 |
| Tawfic Wael Kobtan | 61-12971 | T21 |
| Osama Nassar | 61-35239 | T21 |
| Mohamed Ahmed Enan | 61-34803 | T25 |
| Ziad Abdou Hussien | 61-29587 | T25 |
| Mohamed Shams | 61-15365 | T21 |

## Detailed Project Report

### 1. Introduction

The **Harvard Processor Simulator** is a custom 16-bit processor design implemented to execute a specific subset of instructions adhering to the **Package 4** specifications ("Double McHarvard with cheese circular shifts"). The architecture features a Harvard memory model with separate instruction and data memories, and employs a 3-stage pipeline to enhance instruction throughput. The project comprises a robust C-based backend for memory and execution modeling alongside a JavaScript-based interactive GUI for comprehensive visualization.

### 2. Architecture Details

The simulated processor is built upon the following architectural foundations:

- **Memory Subsystem (Harvard Architecture):**
    - **Instruction Memory:** 1024 words × 16 bits. Stores the encoded binary instructions parsed from the assembly file. It is word-addressable.
    - **Data Memory:** 2048 bytes × 8 bits. Used for data storage and manipulation via load and store operations. It is byte-addressable.
- **Register File:**
    - **General Purpose Registers (GPRs):** 64 registers (R0 to R63), capable of storing 8-bit values.
    - **Status Register (SREG):** An 8-bit register containing 5 condition flags updated by arithmetic and logical operations: `Carry (C)`, `Overflow (V)`, `Negative (N)`, `Sign (S)`, and `Zero (Z)`.
- **Pipeline Structure:**
  A 3-stage pipeline is implemented to optimize execution speed:
    1. **Instruction Fetch (IF):** Fetches the 16-bit instruction from Instruction Memory at the address specified by the Program Counter (PC).
    2. **Instruction Decode (ID):** Decodes the instruction opcode, reads operands from the register file, and sign-extends immediate values.
    3. **Execute (EX):** Performs the requested ALU operation, resolves memory accesses (LB/SB), evaluates branch conditions, and writes results back to the register file.

### 3. Instruction Set Architecture (ISA)

The processor supports two primary instruction formats encoded into a 16-bit word:

- **R-Type Format:** `[Opcode: 4 bits] | [R1: 6 bits] | [R2: 6 bits]`
- **I-Type Format:** `[Opcode: 4 bits] | [R1: 6 bits] | [Immediate: 6 bits]`

**Supported Instructions:**

1. **Arithmetic:** `ADD`, `SUB`, `MUL`
2. **Logical:** `AND`, `OR`
3. **Memory Access:** `LB` (Load Byte), `SB` (Store Byte)
4. **Control Flow:** `BEQZ` (Branch if Equal to Zero), `JR` (Jump Register)
5. **Data Movement:** `LDI` (Load Immediate)
6. **Shift Operations (Package 4 Specific):** `SAL` (Shift Arithmetic Left), `SAR` (Shift Arithmetic Right)

### 4. Hazard Detection and Pipeline Management

To ensure correct execution within the pipelined architecture, the simulator implements comprehensive hazard management:

- **Data Hazards:** The processor detects Read-After-Write (RAW) data dependencies between adjacent instructions. While strict forwarding is supported where applicable, the design ensures proper pipeline stalling or value resolution to maintain accurate register states.
- **Control Hazards:** Handled gracefully during branch (`BEQZ`) and jump (`JR`) operations. The pipeline flushes incorrectly fetched instructions when a branch is taken, dynamically updating the Program Counter (PC) to the correct target address.

### 5. Simulator Implementation

- **Backend (C Language):** Provides the core logic, memory management, and parser functionality. The parser converts human-readable assembly files into the required 16-bit binary format, removing comments and symbolic names, ensuring pure execution simulation.
- **Frontend GUI (JavaScript):** A web-based graphical user interface visualizing the pipeline state on a cycle-by-cycle basis. It accurately tracks the progression of instructions through the IF, ID, and EX stages, enabling users to visually verify data flow, memory alterations, and register updates.

### 6. Conclusion

The implementation of the Harvard Processor Simulator successfully demonstrates the core mechanics of a pipelined processor. By fulfilling the functional requirements of Package 4, including the accurate handling of newly integrated load/store instructions and complex control flow logic, the team has delivered a fully functional educational tool for architectural visualization and assembly execution.